

# Olist Brazilian E-Commerce Analytics

## Technical Project Summary

### 1. Project Overview

I built an end-to-end e-commerce analytics project using the **Olist Brazilian E-Commerce dataset**. The project focused on understanding product and sales performance, customer ordering behaviour, delivery performance, customer reviews, sales trends, and payment behaviour.

I used three main tools for different parts of the project:

- **Python/Pandas** — for exploratory analysis, data investigation, creating derived analytical fields, identifying data-quality issues, and answering the initial business questions.
- **MySQL/SQL** — for relational analysis and independent validation of important findings from Python.
- **Power BI** — for creating the final analytical data model, DAX measures, interactive visuals, slicers, and the five-page dashboard.

The work was structured around six business questions covering category performance, state-level sales, delivery and reviews, sales trends, new versus repeated orders, and factors associated with changes in sales and order volume.

## 2. Datasets Used

I used the following datasets from the **Olist Brazilian E-Commerce dataset**.

### 2.1 **olist\_orders\_dataset.csv**

**What it contains:**

This dataset contains information about customer orders, including order ID, customer ID, order status, purchase timestamp, approval timestamp, delivery timestamps, and estimated delivery date.

**Grain:**

One row represents **one order**.

**Why I used it:**

I used this as the main order-level dataset because it provides the purchase timeline, order status, and delivery information required for order, sales-trend, and delivery analysis.

### 2.2 **olist\_order\_items\_dataset.csv**

**What it contains:**

This dataset contains the individual products included in each order, including order ID, item number, product ID, seller ID, item price, and freight value.

**Grain:**

One row represents **one item within an order**.

**Why I used it:**

I used it to calculate merchandise sales, items sold, category performance, and other item-level sales metrics.

### 2.3 **olist\_customers\_dataset.csv**

**What it contains:**

This dataset contains customer records with customer ID, unique customer ID, ZIP-code prefix, city, and state.

**Grain:**

One row represents **one customer record**.

**Why I used it:**

I used it to connect orders with customer geography and to distinguish individual customer records from repeat purchases made by the same underlying customer.

## 2.4 olist\_products\_dataset.csv

### What it contains:

This dataset contains product-level information, including product ID, product category, and product attributes.

### Grain:

One row represents **one product**.

### Why I used it:

I used it to connect product IDs from Order Items with product categories so that I could analyse sales by category.

## 2.5 olist\_sellers\_dataset.csv

### What it contains:

This dataset contains seller information, including seller ID and seller location.

### Grain:

One row represents **one seller record**.

### Why I used it:

I reviewed this dataset as part of understanding the available Olist data. It was not required for the final six-question analysis or dashboard, so I did not build a separate seller-performance analysis.

## 2.6 olist\_order\_payments\_dataset.csv

### What it contains:

This dataset contains payment transactions for orders, including order ID, payment sequence, payment type, number of installments, and payment value.

### Grain:

One row represents **one payment transaction**.

### Why I used it:

I used it for payment-method, installment, payment-value, and multiple-payment analysis.

## 2.7 olist\_order\_reviews\_dataset.csv

### What it contains:

This dataset contains customer review information, including order ID, review ID, review score, review comments, and review dates.

### Grain:

One row represents **one review record**.

### Why I used it:

I used the review score to investigate the relationship between delivery time and customer experience.

## 2.8 olist\_geolocation\_dataset.csv

### What it contains:

This dataset contains geographic information associated with Brazilian ZIP-code prefixes, including geographic coordinates.

### Grain:

It contains multiple geographic records associated with ZIP-code prefixes.

### Why I used it:

I reviewed it as part of understanding the available source data, but it was not required for the final analysis or Power BI dashboard.

## 2.9 product\_category\_name\_translation.csv

### What it contains:

This dataset maps the original Portuguese product-category names to English category names.

### Grain:

One row represents **one category translation**.

### Why I used it:

I used it to make the category analysis easier to understand by displaying English category names in the analysis and dashboard.

## 3. Python / Pandas — What I Created and Why

I used Python/Pandas as the main analytical environment for exploring the data, creating derived fields, investigating relationships, and producing the initial findings.

### 3.1 Created **actual\_delivery\_time**

I created **actual\_delivery\_time** by calculating the difference between the order purchase timestamp and the actual customer delivery timestamp.

#### Why I created it:

The original dataset contained separate purchase and delivery timestamps. I needed to calculate the actual elapsed delivery duration before I could analyse delivery performance.

### 3.2 Created **delivery\_days**

I converted **actual\_delivery\_time** into a numerical number of days and rounded it to two decimal places.

#### Why I created it:

I needed a numerical delivery measure that could be used to calculate averages, medians, distributions, state-level comparisons, and the relationship between delivery time and review scores.

I found a **median delivery time of 10.22 days** and a **mean of 12.56 days**.

### 3.3 Investigated Delivery Outliers

I calculated the quartiles and IQR of delivery days and identified **29.145 days** as the upper outlier boundary.

#### Why I did this:

I wanted to understand whether unusually long deliveries were strongly influencing the delivery-versus-review relationship.

I did not simply remove these records. I compared the correlation before and after excluding deliveries above the boundary to understand the effect of extreme values.

### 3.4 Created **status\_delivery\_issue**

I created a flag identifying cancelled orders that nevertheless contained a customer delivery date.

**Why I created it:**

This combination of information is inconsistent. I wanted to identify these records as a data-quality issue rather than silently treating them as normal deliveries.

I identified **6 such records**.

### 3.5 Created an Order-Level Review Dataset

I found that some orders appeared more than once in the Reviews dataset.

I therefore aggregated the review score by **order\_id** to create an order-level average review score.

**Why I created it:**

The delivery analysis was performed at order level. I needed one review value per order so that repeated review records would not create duplicated order-level observations when combined with delivery information.

### 3.6 Created **delivery\_bucket**

I grouped delivery times into:

- 0–5 days
- 5–10 days
- 10–15 days
- 15–20 days
- 20–25 days
- 25–29.145 days

**Why I created it:**

I wanted to make the relationship between delivery duration and review scores easier to interpret than using only a single correlation value.

The average review score declined from approximately **4.45** in the 0–5 day group to **3.35** in the 25–29.145 day group.

## 3.7 Created Category Sales Analysis

I joined Order Items with Products and the category translation data.

### Why I created it:

Order Items contains the sales transaction and product ID, while Products contains the category. I needed these datasets together to calculate category-level sales.

I calculated:

- total category sales
- items sold
- average item price
- average sales per order

This allowed me to distinguish high sales generated through **volume** from high sales generated through **higher-value items**.

## 3.8 Created State-Level Delivery Analysis

I combined delivered orders with customer information.

### Why I created it:

I needed customer state information alongside delivery duration to compare delivery performance geographically.

I calculated average delivery days and order counts for each state and used an order-volume threshold when making the state comparison.

## 3.9 Created Monthly Sales Analysis

I combined Order Items with Orders and used the purchase timestamp to group merchandise sales by month.

### Why I created it:

I needed to understand how sales changed over time based on when customers purchased.

I identified **November 2017** as the highest-sales month, with approximately **₹1.01 million** in merchandise sales.

## 3.10 Created New vs Repeated Order Classification

I combined Orders with Customers and used `customer_unique_id` to identify the customer's first purchase.

### **Why I created it:**

I needed to distinguish a customer's first order from subsequent orders to analyse new versus repeated ordering behaviour.

I then compared new and repeated order volumes month by month.

## 3.11 Created Monthly Sales and Order Comparison

I combined monthly sales with monthly order counts.

### **Why I created it:**

I wanted to investigate whether changes in sales were always accompanied by changes in order volume.

This allowed me to identify months where sales increased while order volume decreased, and vice versa, indicating that sales changes could also be associated with changes in the value generated per order.

## 4. SQL / MySQL — What I Created and Why

I used MySQL as a separate relational analysis environment. The purpose was not to repeat every Python calculation, but to perform important analyses through SQL and independently validate key results.

### 4.1 Created **olist\_analytics** Database

I created a dedicated MySQL database named **olist\_analytics**.

#### Why I created it:

I wanted a structured relational environment where I could work with the Olist tables and perform joins, aggregations, CTEs, and window-function analysis using SQL.

### 4.2 Created **category\_sales\_data**

I created a SQL view joining:

- Order Items
- Products
- Category Translation

#### Why I created it:

I needed a reusable SQL dataset connecting each sales transaction with its English product category.

I used this view to calculate total sales, items sold, average item sales, and average sales per order by category.

### 4.3 Created **state\_sales\_data**

I created a SQL view joining:

- Customers
- Orders
- Order Items

#### Why I created it:

I needed to connect sales transactions with customer states so that I could calculate state-level sales, order counts, and average sales per order.

## 4.4 Created `monthly_sales_data`

I created a SQL view containing order purchase timestamps and item prices by joining Orders and Order Items.

### **Why I created it:**

I needed a reusable SQL dataset for monthly sales analysis.

Because the timestamp was imported as text, I used `STR_TO_DATE()` to convert it before grouping the transactions by month.

## 4.5 Created New vs Repeated Order Classification in SQL

I used `ROW_NUMBER()` partitioned by `customer_unique_id` and ordered by purchase timestamp.

### **Why I created it:**

I wanted to independently classify the first order of each customer as new and subsequent orders as repeated.

This allowed me to validate the customer-order classification I had created in Python.

# 5. Power BI

I used Power BI to convert the analytical work into an interactive business dashboard.

This section covers **data modeling, tables created, DAX measures, and dashboard construction.**

## 5.1 Power BI Data Modeling

I designed the Power BI model according to the grain of each table so that calculations would not incorrectly duplicate orders, customers, or payments.

### Customers → Orders

I created a *1: relationship\** from `Customers_PowerBI[customer_id]` to `Orders_PowerBI[customer_id]`.

#### Why I created it:

One customer can have multiple orders, so the customer table is on the one side and the Orders table is on the many side.

### Orders → Order Items

I created a *1: relationship\** from `Orders_PowerBI[order_id]` to `Order_Items_PowerBI[order_id]`.

#### Why I created it:

One order can contain multiple item records.

### Orders → Payments

I created a *1: relationship\** from `Orders_PowerBI[order_id]` to `Payments_PowerBI[order_id]`.

#### Why I created it:

One order can contain multiple payment transactions.

### Orders ↔ Payment Summary

I created a **1:1 relationship** between Orders and `Payment_Summary_PowerBI`.

**Why I created it:**

The payment summary contains exactly one aggregated record per order, allowing order-level payment calculations without the duplication that exists in the raw transaction-level Payments table.

**Orders ↔ Delivery Reviews**

I created a **1:1 relationship** between Orders and **Delivery\_Reviews\_PowerBI**.

**Why I created it:**

The delivery-review table was prepared at order level, so each analysed order has one corresponding record.

## 5.2 Power BI Tables I Created

### Customers\_PowerBI

**Grain:** One customer record.

**Why I created it:**

I needed a clean customer-level table for customer counts and geographic analysis.

### Orders\_PowerBI

**Grain:** One order.

**Why I created it:**

I needed the main order-level table for order counts, purchase dates, delivery information, order classification, and dashboard filtering.

### Order\_Items\_PowerBI

**Grain:** One order-item record.

**Why I created it:**

I needed item-level sales and category information for the product and sales pages.

## Delivery\_Reviews\_PowerBI

**Grain:** One analysed order.

**Why I created it:**

I needed delivery duration, review score, and delivery bucket together at order level for the customer-experience analysis.

## Payments\_PowerBI

**Grain:** One payment transaction.

**Why I created it:**

I needed the original payment-level data for payment method and installment analysis.

## Payment\_Summary\_PowerBI

**Grain:** One order.

**Why I created it:**

I needed order-level payment information for payment KPIs without counting an order multiple times when it had multiple payment transactions.

I created:

- `total_payment_value`
- `payment_count`
- `payment_methods`
- `max_installments`

## 5.3 Supporting Tables I Created

### Day\_Table

I created a separate Day table containing the seven days of the week and their numerical order.

**Why I created it:**

I needed the weekday chart to display Monday through Sunday in the correct sequence rather than alphabetical order.

## Delivery\_Bucket\_Table

I created a separate delivery-bucket table containing the delivery ranges and their sort order.

### Why I created it:

I needed the delivery buckets to appear in numerical order. Creating the separate table also solved the circular-dependency problem I encountered when attempting to create the sorting logic directly within the delivery-review table.

## 5.4 DAX Measures I Created and Why

### Total\_Sales

I created this measure to calculate total merchandise sales from item prices.

#### Why:

I needed one reusable sales measure that could respond dynamically to Power BI filters and slicers.

### Total\_Orders

I created this measure using distinct order IDs.

#### Why:

The Orders table represents orders directly, but I wanted a reusable measure that always counts each order once.

### Total\_Customers

I created this measure using distinct `customer_unique_id`.

#### Why:

I needed to count actual unique customers rather than simply counting customer records.

### Avg\_order\_value

I created this measure by dividing total sales by total orders.

#### Why:

I needed to understand the average merchandise value generated per order.

## Median\_Delivery\_Days

I created this measure to calculate the median delivery duration.

### Why:

Delivery times contain extreme values, so the median provides a useful central measure that is less influenced by unusually long deliveries.

## Average\_Delivery\_Days

I created this measure to calculate average delivery duration.

### Why:

I needed the mean delivery time as a complementary measure to the median.

## Active\_Customers

I created this measure to count unique customers associated with the orders currently in context.

### Why:

I needed the customer KPI on the Executive Overview to respond appropriately to the order-level filtering context.

## New\_Orders

I created this measure to count orders classified as **New**.

### Why:

I needed to separate first-time orders from repeated orders in the customer and order analysis.

## Repeat\_Orders

I created this measure to count orders classified as **Repeated**.

### Why:

I needed a separate KPI and visual measure for repeat-order behaviour.

## Avg\_Review\_Score

I created this measure to calculate the average review score.

### Why:

I needed a consistent measure for comparing customer review scores across delivery-time buckets.

## State\_Order\_Count

I created this measure to count distinct orders by state.

### Why:

I needed it to apply the **300+ order threshold** to the state delivery comparison so that the comparison focused on states with sufficient order volume.

## Payment\_Revenue

I created this measure from the order-level payment summary.

### Why:

I needed a payment-value KPI based on the aggregated payment data.

## Avg\_Payment\_Per\_Order

I created this measure by dividing payment revenue by total orders.

### Why:

I needed to understand the average payment value associated with an order.

## Multiple\_Payment\_Orders

I created this measure to count orders where `payment_count` was greater than one.

### Why:

I wanted to analyse how the number of orders involving multiple payment transactions changed over time.